

Fundamentos de Inteligencia Artificial Clásica: Técnicas, Métodos y Aplicaciones Antes de la IA Generativa

Luis Chamba-Eras

Editor

2026

Índice general

I	Introducción y Fundamentos	1
II	Aprendizaje Estadístico y Modelos de Clasificación	2
1.	Optimización mediante PSO en Ingeniería	3
1.1.	Introducción	3
1.2.	Fundamentos Teóricos de PSO	4
1.3.	Evolución y Variantes del Algoritmo	6
1.3.1.	Topologías de Enjambre	6
1.3.2.	PSO Multiobjetivo (MOPSO)	7
1.3.3.	Hibridaciones y Sinergias Algorítmicas	7
1.4.	Aplicaciones en la Ingeniería	8
1.4.1.	Ingeniería de Computación y Software	8
1.4.2.	Diseño Electrónico y Sistemas Digitales	8
1.4.3.	Sistemas de Energía y Potencia	9
1.4.4.	Procesamiento de Señales y Sistemas Embebidos	9
1.4.5.	Logística, Enrutamiento y Sistemas de Información	10
1.5.	Análisis de Complejidad Computacional	12
1.6.	Conclusiones	13

Parte I

Introducción y Fundamentos

Parte II

Aprendizaje Estadístico y Modelos de Clasificación

Capítulo 1

Optimización de Sistemas de Ingeniería mediante el Algoritmo de Optimización por Enjambre de Partículas (PSO)

Autor: Jaime Alejandro Landázuri Román (jaime.landazuri@unl.edu.ec)

Técnica asignada: Optimización por Enjambre de Partículas (PSO)

1.1. Introducción

La optimización es un proceso central en el diseño, control, operación y manufactura de sistemas de ingeniería [1–3]. El objetivo consiste en identificar parámetros que minimicen costos, maximicen la eficiencia o equilibren objetivos en conflicto [1, 4]. Históricamente, estos problemas se han abordado mediante métodos analíticos, técnicas deterministas y algoritmos basados en el descenso de gradiente [5, 6]. No obstante, la complejidad de las aplicaciones actuales expone limitaciones en estos enfoques, los cuales presentan dificultades para converger en problemas de ingeniería modernos [1, 5, 7]. Estas dificultades se originan principalmente en tres factores: la alta no linealidad del sistema, la topografía multimodal de la función objetivo y la presencia de restricciones [2, 8, 9].

En primer lugar, los problemas en ingeniería suelen presentar **no linealidad** y, con frecuencia, falta de diferenciabilidad [1, 10]. Esto ocurre porque las interacciones entre variables de diseño —sean continuas, discretas o mixtas— generan superficies irregulares que no se simplifican mediante relaciones lineales [8, 11, 12]. Además, el desempeño depende en ocasiones de simulaciones de caja negra, como el análisis de elementos finitos, donde el cálculo de gradientes es inestable o computacionalmente costoso [3, 13]. El aumento en la dimensión de los parámetros a optimizar también conlleva la denominada "maldición de la dimensionalidad", que genera correlaciones no lineales difíciles de manejar para los métodos deterministas tradicionales [6, 13–15].

En segundo lugar, la **multimodalidad** del espacio de soluciones dificulta el desempeño de los esquemas de optimización convencionales [9, 11, 16, 17]. Las funciones multimodales poseen diversos óptimos locales, denominados "valles de engaño", que dificultan la identificación del óptimo global [5, 10]. Los métodos basados en derivadas son sensibles a las condiciones iniciales; si la búsqueda converge hacia un óptimo local, el algoritmo suele detenerse de forma prematura [2, 5, 14, 15, 18, 19]. Para aplicaciones como el control de sistemas no lineales o el diseño aerodinámico, se emplean estrategias de búsqueda que equilibran la exploración del espacio global con la explotación de regiones específicas [9, 20, 21].

Por otra parte, los diseños ingenieriles están **sujetos a restricciones** que limitan el dominio de búsqueda [4, 22, 23]. Estas restricciones se formulan como sistemas de ecuaciones o inecuacio-

nes no lineales que representan límites físicos, geométricos o económicos, tales como tolerancias de esfuerzo, frecuencias de resonancia o deflexiones [2, 6, 20–22]. Estas condiciones alteran la topología del espacio factible, lo que puede invalidar soluciones óptimas desde un punto de vista puramente matemático [21, 24]. En consecuencia, los algoritmos requieren mecanismos de manejo de restricciones o funciones de penalización para descartar configuraciones que no cumplan con los estándares de integridad y seguridad del modelo [4, 18, 22, 23]. Como alternativa a los métodos analíticos tradicionales ante problemas no lineales, multimodales y restringidos, se han adoptado los algoritmos de optimización metaheurística basados en inteligencia de enjambre [2, 5, 11, 25]. Estos enfoques poblacionales no requieren información del gradiente y emplean lógicas estocásticas de autoorganización, lo que facilita la evasión de óptimos locales [3, 8, 26]. Entre estas técnicas destaca el Algoritmo de Optimización por Enjambre de Partículas (PSO), introducido por Kennedy y Eberhart; un esquema heurístico utilizado por su simplicidad algorítmica, facilidad de paralelización y su capacidad para combinar la experiencia cognitiva individual con la interacción social del grupo para alcanzar soluciones globales de forma iterativa [1, 3, 16, 21, 25, 27].

Este trabajo analiza la mecánica interna de esta metaheurística a través del estudio de sus vectores de velocidad, la estructura de sus redes sociales y las estrategias de ajuste de hiperparámetros, con el objetivo de detallar cómo el algoritmo PSO resuelve problemas complejos en el ámbito de la ingeniería.

Para estructurar esta respuesta, el resto del capítulo guiará al lector a través de un desglose integral de esta metaheurística. En la Sección 1.2, se exponen los fundamentos teóricos y las ecuaciones matemáticas elementales que rigen la cinemática estocástica de las partículas. Posteriormente, la Sección 1.3 aborda la evolución algorítmica del PSO para mitigar sus deficiencias clásicas, explorando topologías de red, enfoques multiobjetivo (MOPSO) y arquitecturas híbridas. La Sección 1.4 demuestra la extraordinaria resiliencia de la técnica al detallar sus aplicaciones prácticas contemporáneas en disciplinas críticas como la ingeniería de software, el diseño electrónico, la logística y los sistemas de potencia. Seguidamente, la Sección 1.5 evalúa la viabilidad de su implementación mediante un riguroso análisis de complejidad computacional y estrategias modernas de paralelización en alta dimensión. Finalmente, la Sección 1.6 consolida las conclusiones sobre el papel indispensable e indiscutible de esta inteligencia colectiva en la vanguardia del diseño ingenieril.

1.2. Fundamentos Teóricos de PSO

El Algoritmo de Optimización por Enjambre de Partículas (PSO), en su formulación clásica, opera mediante la simulación de la dinámica social de un enjambre donde cada individuo (partícula) representa una solución candidata dentro de un espacio de búsqueda de D dimensiones [1, 3, 21]. El desplazamiento de las partículas hacia regiones óptimas se realiza a través de un proceso iterativo basado en dos ecuaciones fundamentales que actualizan el vector de velocidad $v_i(t)$ y el vector de posición $x_i(t)$ de la i -ésima partícula [13, 26, 28]. Estas ecuaciones se definen como:

$$v_i(t+1) = \omega v_i(t) + c_1 r_1 (pbest_i - x_i(t)) + c_2 r_2 (gbest - x_i(t)) \quad (1.1)$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (1.2)$$

En esta formulación, la convergencia y la trayectoria estocástica del enjambre dependen del equilibrio dinámico regulado por los hiperparámetros del sistema [2, 5]. Los coeficientes de aceleración, c_1 (cognitivo) y c_2 (social), ponderan la influencia de la memoria individual frente a la información colectiva del enjambre [1, 21, 29]. Un valor elevado de c_1 ocasiona que las partículas busquen de manera aislada en torno a sus mejores posiciones históricas, lo que incrementa la diversidad pero retrasa la convergencia global [21, 29]. Por el contrario, un valor predominante de

c_2 ejerce una fuerte atracción hacia el óptimo global actual (gbest), acelerando la convergencia a riesgo de inducir un estancamiento prematuro en óptimos locales [13, 29]. Para introducir componentes estocásticos, las variables r_1 y r_2 actúan como vectores aleatorios con distribución uniforme en el intervalo $[0,1]$ [1, 3].

El parámetro principal para estabilizar este sistema dinámico es el peso de inercia (ω). Este coeficiente determina la conservación de la velocidad previa, actuando como un regulador entre la exploración del espacio global y la explotación de regiones locales [1, 13, 30]. Un valor de ω alto al inicio del proceso otorga el momento necesario para que el enjambre evite óptimos locales y explore el espacio de búsqueda. Conforme avanzan las iteraciones, este valor disminuye para permitir un refinamiento localizado en las proximidades de la mejor solución encontrada [7, 8, 15]. Para modelar este comportamiento, se emplea una estrategia de decaimiento lineal, formalizada mediante la siguiente ecuación:

$$\omega(t) = \omega_{max} - \frac{\omega_{max} - \omega_{min}}{t_{max}} \times t \quad (1.3)$$

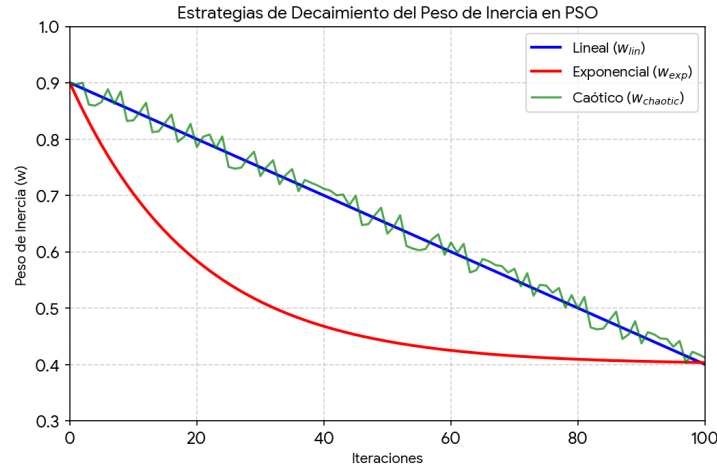


Figura 1.1: Estrategias de variación temporal para el decaimiento del peso de inercia (ω) en PSO a lo largo de 100 iteraciones. Se ilustra la transición desde una fase de exploración agresiva ($\omega = 0,9$) hacia una de explotación microscópica ($\omega = 0,4$). Además del decaimiento lineal clásico formulado en la Ecuación (1.3), se contrastan enfoques avanzados como el decaimiento exponencial (que acelera la transición hacia la explotación) y el mapeo caótico (que inyecta estocasticidad para evitar óptimos locales).

Donde ω_{max} (típicamente 0,9) y ω_{min} (típicamente 0,4) representan los límites superior e inferior del momento de inercia, t es la iteración actual y t_{max} es el límite máximo de iteraciones [7, 21, 30]. Este decaimiento asegura una metamorfosis fluida y matemáticamente estable desde un régimen de exploración agresiva hacia uno de explotación microscópica, mitigando sustancialmente las inestabilidades oscilatorias [15, 30].

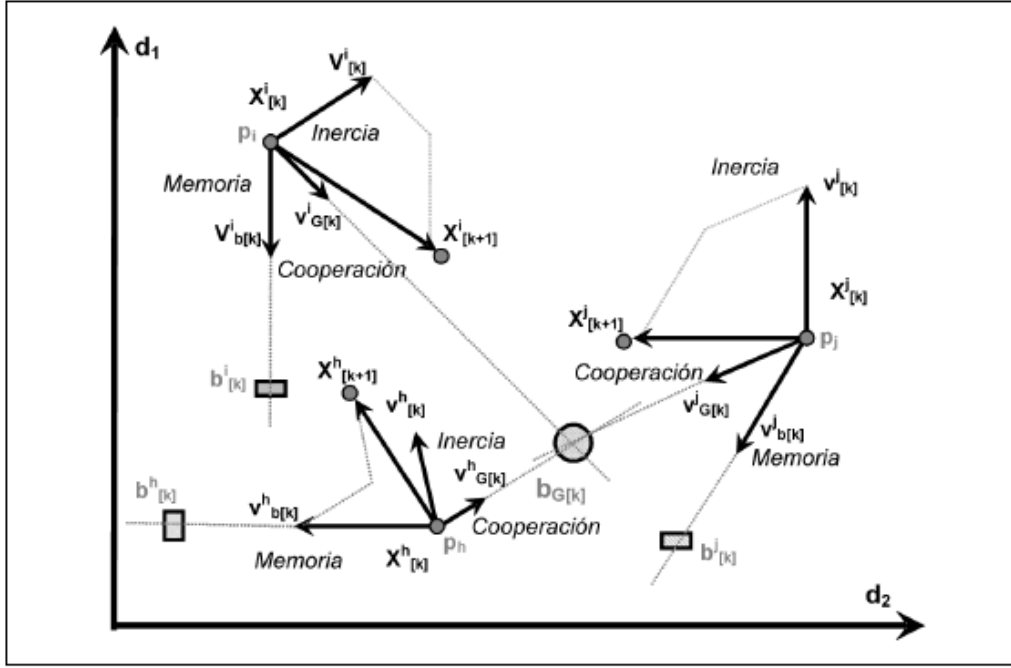


Figura 1.2: Representación vectorial de la actualización cinemática de las partículas en un espacio bidimensional (d_1, d_2). Nótese la equivalencia de notación con las Ecuaciones (1.1) y (1.2): el instante actual se denota como k (equivalente a t), el óptimo personal o vector de memoria es $b^i_{[k]}$ (equivalente a $pbest_i$), y el óptimo global o vector de cooperación es $b^g_{[k]}$ (equivalente a $gbest$) [31].

1.3. Evolución y Variantes del Algoritmo

A pesar de su diseño estructural, el algoritmo PSO clásico presenta limitaciones en paisajes de optimización multimodales y no lineales, manifestadas principalmente en la convergencia prematura o estancamiento [1,5,14]. Al incrementar la dimensionalidad del problema, el enjambre se ve afectado por la denominada "maldición de la dimensionalidad", donde la rápida propagación de la información de un óptimo local reduce la diversidad del sistema y detiene la búsqueda de forma ineficiente [13,15,21]. Para mitigar estas restricciones, se han desarrollado diversas modificaciones arquitectónicas en el algoritmo.

1.3.1. Topologías de Enjambre

La velocidad de transmisión de los datos depende directamente de la estructura de la red social o "topología de vecindario" del enjambre [21,25]. En la topología de *estrella* (o modelo *gbest*), todas las partículas están interconectadas, lo que permite que la información de la mejor solución se comparta con toda la población; esto incrementa la velocidad de convergencia, pero eleva la probabilidad de estancamiento en óptimos locales [15,21,25]. Por el contrario, la topología en *anillo* (modelo *lbest*) restringe la comunicación a partículas adyacentes; esta configuración ralentiza la propagación de los datos, lo que favorece la exploración del espacio de búsqueda y preserva la diversidad de la población [13,21]. Otras estructuras, como la topología de *von Neumann* (un retículo bidimensional), conectan a cada partícula con cuatro vecinos (arriba, abajo, izquierda y derecha), proporcionando un balance intermedio entre la tasa de exploración y el tiempo de cómputo requerido [13,21,25].

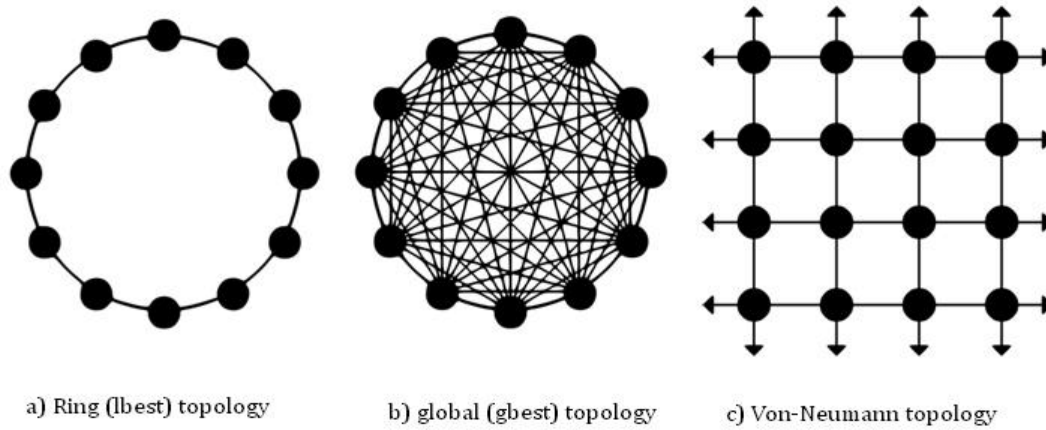


Figura 1.3: Representación visual de las topologías de vecindario fundamentales que estructuran la red social en PSO: a) Topología en anillo o local (*lbest*), que restringe la comunicación para promover una exploración exhaustiva y conservar la diversidad; b) Topología global o de estrella (*gbest*), donde la interconexión total maximiza la velocidad de convergencia; y c) Topología de von Neumann en cuadrícula, que proporciona un equilibrio empírico superior entre exploración y explotación [32].

1.3.2. PSO Multiobjetivo (MOPSO)

En la ingeniería actual, los problemas suelen requerir la optimización simultánea de múltiples objetivos en competencia [1, 27]. Por ejemplo, en el diseño de componentes aeronáuticos, es necesario maximizar la resistencia estructural y la rigidez mecánica, al tiempo que se reduce el peso del material y los costos de manufactura. Para abordar estos escenarios se utiliza el PSO Multiobjetivo (MOPSO), una adaptación que sustituye la búsqueda de un único óptimo global por el principio de dominancia de Pareto [3, 27]. En MOPSO, el algoritmo gestiona un repositorio externo de soluciones no dominadas, denominado Frente de Pareto, desde el cual se seleccionan los guías globales; esto permite evaluar un conjunto de soluciones de compromiso ("trade-offs") para la toma de decisiones [3, 33].

1.3.3. Hibridaciones y Sinergias Algorítmicas

La limitación del algoritmo PSO en las etapas finales de búsqueda se debe a la ausencia de operadores de salto probabilístico o de mutación evolutiva, lo que motiva el desarrollo de arquitecturas híbridas [1, 3, 21]. Una alternativa orientada a sistemas dinámicos no lineales es la combinación con el Optimizador de Lobo Gris (PSO-GWO). En este enfoque, el algoritmo GWO aporta una estructura de exploración basada en una jerarquía social (lobos α, β, δ) para identificar diferentes regiones del espacio de búsqueda durante las fases iniciales e intermedias. Por su parte, el PSO se utiliza para intensificar la búsqueda y acelerar la explotación en las zonas prometedoras identificadas, disminuyendo el número de evaluaciones requeridas para la función de costo [9].

De igual forma, la hibridación con Algoritmos Genéticos (GA) introduce operadores estocásticos de *crossover* y *mutación* dentro del ciclo de actualización del PSO, lo que aporta diversidad a la población y contribuye a modificar las posiciones estancadas [1, 3, 19]. Adicionalmente, la integración con la metodología de Recocido Simulado (SA) permite a las partículas aceptar soluciones de menor calidad bajo un criterio de probabilidad probabilístico; este mecanismo facilita la salida de mínimos locales y asiste en la resolución de funciones con superficies complejas [1, 3].

1.4. Aplicaciones en la Ingeniería

El algoritmo PSO se utiliza como un método de resolución en diversas ramas de la ingeniería aplicada debido a sus propiedades operativas. Al trabajar en espacios de búsqueda continuos y discretos de alta dimensionalidad, y al prescindir de gradientes derivables, este algoritmo facilita la convergencia hacia óptimos globales en entornos con ruido, multimodalidad y restricciones [1, 3, 21]. A continuación, se describen sus aplicaciones en diferentes disciplinas de la ingeniería.

1.4.1. Ingeniería de Computación y Software

En la ingeniería de computación y el desarrollo de arquitecturas de software, el algoritmo PSO se utiliza para la optimización de infraestructuras tecnológicas y el procesamiento de datos. Una de sus aplicaciones se encuentra en el ajuste de hiperparámetros para modelos de *Machine Learning* y arquitecturas de redes neuronales profundas (DNNs y CNNs). Mediante la exploración del espacio de búsqueda, PSO determina tasas de aprendizaje, coeficientes de regularización, factores de penalización en Máquinas de Vectores de Soporte (SVM) y la cantidad de unidades en capas ocultas; esto permite mejorar la precisión de clasificación y disminuir el sobreajuste y los tiempos de entrenamiento en comparación con los métodos de búsqueda en cuadrícula (*Grid Search*) o aleatoria [1, 34, 35].

Simultáneamente, la metaheurística aborda con éxito desafíos de asignación de recursos en entornos de infraestructura como código. En clústeres de servidores y entornos distribuidos de contenedores (*Cloud Computing* y *Edge Computing*), algoritmos como el PSO Modificado (MAL-PSO) y otras variantes híbridas ejecutan un balanceo de carga algorítmico altamente eficiente. Esto minimiza el tiempo de finalización (*makespan*), asegura la estabilidad del sistema y maximiza la eficiencia en el uso de los recursos computacionales virtuales [1, 21]. Asimismo, en ecosistemas de *Big Data* y bases de datos distribuidas o no relacionales, PSO se emplea en la optimización de consultas complejas y la selección de características (*Feature Selection*) en espacios de alta dimensionalidad. Al descartar heurísticamente variables redundantes y optimizar las rutas de acceso a los datos, PSO logra reducir significativamente la latencia de las transacciones y los márgenes de error, garantizando una topología de red de datos depurada y de alta velocidad [1, 21, 36].

1.4.2. Diseño Electrónico y Sistemas Digitales

El diseño electrónico y la automatización de sistemas digitales requieren el cumplimiento de tolerancias estrictas y respuestas transitorias precisas, escenarios donde se aplica el algoritmo PSO. A nivel de control de hardware industrial y robótico, este algoritmo se utiliza para la sintonización de parámetros en controladores Proporcional-Integral-Derivativo (PID) clásicos y de orden fraccionario (FOPID). A través de la minimización iterativa de índices de desempeño integral, como el ITAE o el error cuadrático medio, el PSO —especialmente al combinarse con métodos como el Optimizador de Lobo Gris (PSO-GWO)— se emplea para estabilizar sistemas de control no lineales, tales como osciladores caóticos, reactores químicos continuos (CSTR), sistemas de suspensión activa y péndulos invertidos. Este enfoque contribuye a atenuar el sobreimpulso, disminuir el error en estado estacionario y reducir los tiempos de estabilización en comparación con metodologías tradicionales como la de Ziegler-Nichols [9, 20, 30, 33, 37, 38].

En la arquitectura de hardware, el algoritmo PSO interviene en la síntesis y el diseño de componentes físicos, donde se utiliza para la optimización geométrica y el enrutamiento de alta densidad en placas de circuito impreso (PCB) con el objetivo de reducir las interferencias electromagnéticas cruzadas. En el procesamiento digital de señales y el diseño de transceptores, la variante de PSO de comportamiento cuántico y las hibridaciones mutacionales se emplean para la calibración de filtros digitales de respuesta al impulso finita e infinita (FIR/IIR), la ecualización de canales adaptativos y el diseño de perfiles de códigos polifásicos para radares,

lo que permite disminuir el margen de error y mejorar la fidelidad de la señal en redes de comunicación MIMO-OFDM [16, 21].

1.4.3. Sistemas de Energía y Potencia

La gestión de los recursos energéticos a gran escala plantea desafíos logísticos en los que se aplican soluciones basadas en el algoritmo PSO. El sector eléctrico emplea variantes de este algoritmo, como el PSO Ortogonal y adaptativo, para resolver el problema del Despacho Económico de Energía (ED). Esta tarea consiste en la asignación de las cargas de demanda entre múltiples unidades de generación térmica con el objetivo de disminuir los costos operativos del combustible. El algoritmo PSO aborda este entorno no convexo resolviendo inecuaciones de la red, tales como los límites de tasa de rampa (RRLs), las zonas de operación prohibida (POZs) de los generadores y las pérdidas de transmisión, con un costo computacional menor en comparación con algunos esquemas de programación lineal [1, 14, 21].

En la integración de las redes eléctricas inteligentes (*Smart Grids*), la incorporación de generación distribuida añade factores estocásticos a la matriz energética. En este contexto, el algoritmo PSO se utiliza para determinar la ubicación y el dimensionamiento de generadores basados en energías renovables, como arreglos fotovoltaicos y turbinas eólicas, junto con sistemas de almacenamiento en baterías. Al optimizar los parámetros de la red distribuida, el algoritmo busca incrementar la inyección de energía renovable, disminuir la dependencia de la red principal, mejorar el perfil de tensión en los nodos y reducir las pérdidas activas del sistema para su viabilidad económica y operativa [1, 21].

1.4.4. Procesamiento de Señales y Sistemas Embebidos

En el diseño de hardware y la electrónica, el algoritmo PSO y sus variantes de variables mixtas (MVOPs) o discretas (MDPSO) se utilizan para la sintonización paramétrica de sistemas físicos continuos y redes lógicas [1, 8]. En la instrumentación y el diseño de hardware analógico, la formulación matemática de un circuito requiere seleccionar configuraciones específicas a partir de catálogos comerciales limitados, lo que define un problema de búsqueda no convexa y multi-dimensional [8]. El algoritmo PSO aborda este desafío mapeando los valores de los componentes físicos en el vector de posición x_i de cada partícula, lo que permite explorar espacios de búsqueda continuos o discretos para identificar combinaciones que presentan dificultades con los métodos analíticos convencionales [1, 12].

Un ejemplo de aplicación se encuentra en el desarrollo de circuitos de audio, como el diseño de pedales de efectos analógicos. En este escenario, la metaheurística se emplea para la selección de componentes pasivos y activos, orientando el vector de posición hacia especificaciones comerciales con tolerancias definidas, tales como capacitores electrolíticos, amplificadores operacionales tipo TL072CP o potenciómetros lineales de 10K [3, 21]. La validación del circuito se realiza mediante una función de evaluación (*fitness*) que mide el rendimiento dinámico del sistema; el algoritmo examina el espacio de diseño para disminuir el error cuadrático o absoluto entre la curva de filtrado del hardware propuesto y una respuesta de frecuencia de referencia, ajustando parámetros como la atenuación, la ganancia de banda y el desfase de la señal [16, 21].

En el ámbito de la Síntesis Lógica y los Sistemas Embebidos Digitales, las variantes binarias (BPSO) y discretas (DPSO) de la metaheurística se aplican en problemas de optimización combinatoria inherentes a la automatización del diseño electrónico (EDA) [3, 21]. A diferencia del hardware analógico, en la descripción de hardware estructurado mediante lenguajes como VHDL, el vector de posición de la partícula codifica transiciones binarias o topologías de red que representan los estados de asignación de compuertas lógicas. De esta manera, el algoritmo PSO se utiliza para la asignación de recursos, optimizando el enrutamiento de pistas conductoras y la síntesis de componentes para circuitos secuenciales de alta densidad [3].

Esta técnica se emplea también durante la optimización de módulos digitales desarrollados e implementados en plataformas de arreglos de compuertas programables (FPGAs) como Xilinx Vivado. A medida que las partículas actualizan su velocidad basadas en sus componentes cognitivos y sociales, el algoritmo descarta estructuras redundantes para cumplir con las restricciones físicas del silicio. Consecuentemente, el proceso busca disminuir el área de silicio consumida por el circuito, mitigar el consumo de potencia de conmutación dinámica en los transistores y reducir el retardo de propagación de las señales a lo largo de las rutas críticas del sistema [3, 21]. La convergencia de PSO en estos entornos discretos se orienta al desarrollo de plataformas de procesamiento de señales y arquitecturas digitales que mejoren la eficiencia energética y espacial en la ingeniería de sistemas embebidos.

1.4.5. Logística, Enrutamiento y Sistemas de Información

La gestión logística y la telemetría en tiempo real plantean desafíos computacionales en los que se aplican las propiedades de convergencia del Algoritmo de Optimización por Enjambre de Partículas (PSO). En el contexto de redes de recolección de datos logísticos y sistemas del Internet de las Cosas (IoT), el PSO se utiliza para optimizar las rutas de transmisión de información con el objetivo de disminuir la latencia de la red [1, 21]. Una aplicación específica se encuentra en la configuración y sincronización de sistemas de rastreo GPS para la recepción de tramas provenientes de dispositivos de telemetría de alta frecuencia, como los equipos de la arquitectura Teltonika. Al modelar la topología de la red de transmisión, las partículas del enjambre evalúan el espacio de soluciones para identificar rutas de acceso que reduzcan la latencia en la entrega de *payloads* telemétricos [3, 21].

Simultáneamente, el algoritmo PSO se implementa para resolver las variaciones dinámicas del Problema de Enrutamiento de Vehículos (VRP). Mediante la actualización de sus vectores de velocidad y posición, esta metaheurística busca evitar óptimos locales y coordinar el flujo de datos telemétricos hacia los servidores centrales, disminuyendo la formación de cuellos de botella en la red de comunicaciones [1, 3].

A nivel de infraestructura de software, la integración de PSO en arquitecturas *backend* se utiliza para la orquestación y optimización de recursos computacionales distribuidos. En ecosistemas basados en microservicios y entornos contenerizados con plataformas como Docker, así como en despliegues sobre infraestructuras de nube de alta disponibilidad (tales como Google Cloud Platform y DigitalOcean), el PSO ejecuta el balanceo de carga y la programación de tareas (*task scheduling*) [1, 3, 21]. Esta metaheurística se integra con arquitecturas *backend* controladas por eventos y no bloqueantes en Node.js, e interactúa con sistemas de bases de datos no relacionales como MongoDB. Al aplicar la búsqueda estocástica para la selección de nodos y el enrutamiento de consultas transaccionales, el algoritmo busca minimizar el tiempo de finalización global (*makespan*) y mejorar la escalabilidad de las bases de datos frente a peticiones concurrentes [1, 3, 36].

Estas optimizaciones de latencia de red y enrutamiento en las capas del *backend* influyen en la eficiencia de los flujos de trabajo administrativos. La capacidad de PSO para equilibrar la exploración y la explotación en tiempos reducidos facilita que la transferencia de datos hacia las interfaces de usuario se realice de forma fluida [5, 13]. En el contexto del desarrollo de software, esto se asocia con una disminución de los tiempos de respuesta del servidor y en la actualización de paneles de control web y *dashboards* administrativos. Consecuentemente, la variación asíncrona de información —como las métricas en vivo de una flota logística o el estado de consumo de la infraestructura en la nube— opera en tiempo real, empleando la inteligencia de enjambre para el desarrollo de sistemas de información funcionales [1, 21].

Tabla 1.1: Resumen de la revisión de literatura: Evolución algorítmica, variantes y aplicaciones contemporáneas del Algoritmo de Optimización por Enjambre de Partículas (PSO) en el modelado y diseño de sistemas complejos en ingeniería.

Referencia	Variante de PSO	Problema de Ingeniería	Ventaja Competitiva
[1]	MALPSO (Modified PSO) e híbridos	Balanceo de carga algorítmico y programación de tareas en clústeres de servidores en la nube (Cloud Computing).	Acorta drásticamente el tiempo de finalización (makespan) y equilibra dinámicamente las cargas, optimizando la asignación de recursos virtuales.
[14]	OPSO (Orthogonal PSO)	Despacho Económico de Energía Térmica bajo restricciones no convexas (límites de rampa y zonas prohibidas).	Mitiga el colapso por "zigzagging" usando vectores de diagonalización ortogonal, asegurando convergencia global y reduciendo el costo total del sistema eléctrico.
[22]	SEPSO (Social Engineering PSO)	Diagnóstico inverso de daños estructurales (VBDI) basado en el análisis de variables de vibración mecánica.	Emplea estrategias elitistas y operadores probabilísticos de ataque social para esquivar óptimos locales en dominios topológicos sumamente fragmentados y ruidosos.
[9]	PSO-GWO (Híbrido)	Sintonización algorítmica de parámetros de controladores PID robustos en sistemas dinámicos no lineales.	Reduce a menos del 10 % las evaluaciones de la función de costo al compensar la pérdida de diversidad del PSO con la jerarquía estocástica de exploración del GWO.
[5]	VPPSO (Velocity Pausing PSO)	Diseño ingenieril integral de componentes sujetos a restricciones físicas (ej. deflexión en vigas y tensión en resortes).	Introduce una "pausa de velocidad" probabilística y topología de doble enjambre para estabilizar el equilibrio entre exploración y explotación en problemas de hiper-dimensionalidad.
[2]	TPOPSOGWO (Dos Fases + OBL)	Resolución de hiper-superficies geométricas en problemas de diseño industrial altamente restringidos matemáticamente.	Mejora radicalmente la heterogeneidad de la población al inyectar inicializaciones por Aprendizaje Basado en Oposición (OBL), acelerando la tasa de convergencia sin violar límites físicos.
Continúa en la siguiente página...			

Tabla 1.1 – Continúa de la página anterior

Referencia	Variante de PSO	Problema de Ingeniería	Ventaja Competitiva
[6]	PSO-HS (con Harmony Search) y EPSO	Análisis de Confiabilidad Estructural de Primer Orden (FORM) y búsqueda de puntos de falla más probables (MPP).	Elimina la dependencia de vectores de sensibilidad de métodos analíticos de gradiente, brindando invulnerabilidad ante funciones de rendimiento discretas de alta dimensión.
[37]	PSO Clásico (Optimización Bayesiana)	Determinación óptima de ganancias para controladores de modo deslizante (SMC) en manipuladores robóticos (2-DOF).	Encuentra compromisos excepcionales entre errores de estado estacionario y velocidad de respuesta a través de funciones de costo sustitutas (Surrogate Models), superando enfoques evolutivos estándar.
[19]	PSO con Mutación de Ruido Blanco (GWN)	Actualización dinámica de modelos de elementos finitos (FEA) de alta fidelidad para puentes atirantados a gran escala.	Inyecta ruido blanco gaussiano como mecanismo mutacional para provocar saltos estocásticos de fuga y evadir el estancamiento prematuro en sistemas de ecuaciones altamente acoplados.

1.5. Análisis de Complejidad Computacional

El análisis de la complejidad computacional del Algoritmo de Optimización por Enjambre de Partículas (PSO) es necesario para evaluar su escalabilidad en problemas de ingeniería. Con base en la notación asintótica Big-O, el límite superior de crecimiento temporal del algoritmo clásico se define mediante la expresión $O(NID)$, donde N representa el tamaño del enjambre (número de partículas), I equivale al límite de iteraciones máximas para la convergencia, y D denota la dimensionalidad del espacio de búsqueda [18, 30]. Desde una perspectiva analítica, el costo computacional principal de PSO no reside en la actualización de los vectores de velocidad y posición en cada iteración —operación algebraica lineal de orden $O(ND)$ —, sino en la evaluación iterativa de la función objetivo o *función fitness* [18, 30]. Conforme aumenta la complejidad del modelo a optimizar (como en el análisis de elementos finitos o la dinámica de fluidos computacional), el tiempo de procesamiento requerido por el simulador para determinar la calidad de las soluciones representa la mayor parte de la carga de cómputo, superando la demanda matemática propia del algoritmo estocástico [13, 30]. Este comportamiento se acentúa al evaluar el costo computacional de las hibridaciones y variantes avanzadas. Si bien la integración de arquitecturas modificadas es necesaria para evitar deficiencias como el atrapamiento en óptimos locales, esto acarrea un compromiso (*trade-off*) que incrementa la carga de procesamiento del sistema [1, 30]. Por ejemplo, el desarrollo del PSO Multiobjetivo (MOPSO) añade complejidad debido a la necesidad de comparar, clasificar y ordenar las soluciones no dominadas frente al resto de la población para mantener actualizado el repositorio externo del Frente de Pareto [3, 27]. Del mismo modo, las hibridaciones como la arquitectura PSO-GWO incrementan la cantidad de cálculos por iteración, al requerir que la población se someta simultáneamente

a la lógica jerárquica de exploración del optimizador de lobo gris y al esquema de explotación posicional basado en el peso de inercia del PSO [9]. A partir del análisis de complejidad, es posible evaluar la viabilidad de implementación del algoritmo. El PSO y sus variantes híbridas se utilizan en escenarios de *diseño offline*, simulación de pre-lanzamiento, diseño asistido por computadora (CAD) y el entrenamiento de modelos matemáticos. En estas aplicaciones, un tiempo de ejecución de minutos u horas es admisible para alcanzar una sintonización de parámetros o un diseño geométrico óptimo antes de su despliegue [9, 30]. Sin embargo, este costo computacional plantea limitaciones cuando se intenta integrar la metaheurística dentro del lazo cerrado de sistemas de control dinámico en tiempo real, donde la respuesta ante perturbaciones requiere latencias de resolución en el orden de los milisegundos [30, 33]. Para mitigar los cuellos de botella en problemas de alta complejidad y dimensionalidad, se ha transitado de la ejecución secuencial clásica hacia entornos distribuidos. La gestión actual recurre a la paralelización masiva de las partículas del enjambre sustentándose en el paradigma de "divide y vencerás" [1, 3]. Al independizar la trayectoria de cada individuo, la evaluación simultánea de la función *fitness* se delega a entornos de aceleración por hardware de alto desempeño, empleando infraestructuras como clústeres de múltiples núcleos CPU en la nube o el procesamiento paralelo en Unidades de Procesamiento Gráfico (GPUs) mediante tecnologías como CUDA; esto reduce los tiempos globales de ejecución para viabilizar diseños de gran magnitud computacional [1, 3].

1.6. Conclusiones

A lo largo de este trabajo, se ha analizado cómo el Algoritmo de Optimización por Enjambre de Partículas (PSO) se utiliza como una herramienta matemática y computacional para la resolución de problemas en ingeniería. La evolución desde su formulación clásica hacia arquitecturas híbridas y variantes modificadas ha permitido abordar sus limitaciones operativas. Específicamente, desarrollos como el Algoritmo Crazy PSO con mutación adaptativa y las variantes cooperativas y adaptativas (ACPSO) se emplean para mitigar la convergencia prematura en espacios de búsqueda de alta dimensionalidad [1, 21, 39]. Estas modificaciones estructurales aportan diversidad a la población e introducen componentes estocásticos que facilitan la evasión de óptimos locales en topologías complejas [3, 10].

Con base en el análisis desarrollado, es posible dar respuesta a la interrogante central de esta investigación: ¿Cómo optimiza PSO problemas complejos en ingeniería? El PSO optimiza mediante un equilibrio dinámico entre la exploración del espacio global y la explotación localizada de las regiones prometedoras [1, 16, 18]. Este comportamiento se logra mediante la adaptación de sus parámetros, tales como las topologías de vecindario, el ajuste temporal del peso de inercia y los coeficientes cognitivo y social, permitiendo al algoritmo operar ante las restricciones y no linealidades de los sistemas de ingeniería modernos [3, 13, 21]. Al prescindir de los requerimientos de derivabilidad y fundamentar su comportamiento en la inteligencia colectiva, el enjambre procesa e identifica las interacciones de las variables de diseño de manera iterativa [2, 8].

En conclusión, el uso de PSO en los entornos académico e industrial se mantiene vigente debido a su viabilidad operativa. Su flexibilidad para integrarse con otros optimizadores metaheurísticos y técnicas de inteligencia artificial permite su aplicación en escenarios dinámicos, con ruido o de gran escala. Conforme la disciplina ingenieril se orienta hacia el modelado de sistemas físicos con un mayor número de variables, el PSO se establece como una opción para el diseño automatizado y la optimización integral de sistemas [1, 3, 21].

Referencias

- [1] L. Abualigah, “Particle swarm optimization: Advances, applications, and experimental insights,” *Computers, Materials and Continua*, vol. 82, pp. 1539–1592, 2 2025.
- [2] R. Katarya, U. Agrawal, and V. Rohatgi, “Optimizing constrained engineering problems using opposition based swarm algorithms,” in *Proceedings – 2021 3rd International Conference on Advances in Computing, Communication Control and Networking, ICAC3N 2021*. Institute of Electrical and Electronics Engineers Inc., 12 2021, pp. 861–867.
- [3] Y. Zhang, S. Wang, and G. Ji, “A comprehensive survey on particle swarm optimization algorithm and its applications,” *Mathematical Problems in Engineering*, vol. 2015, p. 931256, 1 2015.
- [4] M. Khorsand and Y. Tang, “Design functionally graded rotating disks under thermoelastic loads: Weight optimization,” *International Journal of Pressure Vessels and Piping*, vol. 161, pp. 33 – 40, 2018, cited by: 40. [Online]. Available: <https://www.scopus.com/pages/publications/85042197178?origin=resultslist>
- [5] T. M. Shami, S. Mirjalili, Y. Al-Eryani, K. Daoudi, S. Izadi, and L. Abualigah, “Velocity pausing particle swarm optimization: a novel variant for global optimization,” *Neural Computing and Applications*, vol. 35, no. 12, pp. 9193 – 9223, 2023, cited by: 138; All Open Access; Green Open Access; Hybrid Gold Open Access. [Online]. Available: <https://www.scopus.com/pages/publications/85145706717?origin=resultslist>
- [6] S.-P. Zhu, B. Keshtegar, M. E. A. Ben Seghier, E. Zio, and O. Taylan, “Hybrid and enhanced pso: Novel first order reliability method-based hybrid intelligent approaches,” *Computer Methods in Applied Mechanics and Engineering*, vol. 393, 2022, cited by: 121. [Online]. Available: <https://www.scopus.com/pages/publications/85125233165?origin=resultslist>
- [7] Y. Shafahi and M. Bagherian, “A customized particle swarm method to solve highway alignment optimization problem,” *Computer-Aided Civil and Infrastructure Engineering*, vol. 28, no. 1, pp. 52 – 67, 2013, cited by: 165. [Online]. Available: <https://www.scopus.com/pages/publications/84871011304?origin=resultslist>
- [8] S. Chowdhury, W. Tong, A. Messac, and J. Zhang, “A mixed-discrete particle swarm optimization algorithm with explicit diversity-preservation,” *Structural and Multidisciplinary Optimization*, vol. 47, no. 3, pp. 367 – 388, 2013, cited by: 114. [Online]. Available: <https://www.scopus.com/pages/publications/84880569625?origin=resultslist>
- [9] R. Vrabel, “Hybrid particle swarm and grey wolf optimization for robust feedback control of nonlinear systems,” *Automation*, vol. 6, p. 89, 12 2025.
- [10] Z. Cui, X. Cai, J. Zeng, and G. Sun, “Particle swarm optimization with fss and rws for high dimensional functions,” *Applied Mathematics and Computation*, vol. 205, no. 1, pp. 98 – 108, 2008, cited by: 51. [Online]. Available: <https://www.scopus.com/pages/publications/54049084147?origin=resultslist>

- [11] A. H. Gandomi, G. J. Yun, X.-S. Yang, and S. Talatahari, "Chaos-enhanced accelerated particle swarm optimization," *Communications in Nonlinear Science and Numerical Simulation*, vol. 18, no. 2, pp. 327 – 340, 2013, cited by: 371. [Online]. Available: <https://www.scopus.com/pages/publications/84866000386?origin=resultslist>
- [12] G. Li and H. Hu, "Risk design optimization using many-objective evolutionary algorithm with application to performance-based wind engineering of tall buildings," *Structural Safety*, vol. 48, pp. 1 – 14, 2014, cited by: 42. [Online]. Available: <https://www.scopus.com/pages/publications/84894294846?origin=resultslist>
- [13] J. Fan, J. Wang, and M. Han, "Cooperative coevolution for large-scale optimization based on kernel fuzzy clustering and variable trust region methods," *IEEE Transactions on Fuzzy Systems*, vol. 22, no. 4, pp. 829 – 839, 2014, cited by: 46. [Online]. Available: <https://www.scopus.com/pages/publications/84905572315?origin=resultslist>
- [14] L. T. Al Bahrani and J. C. Patra, "Orthogonal pso algorithm for economic dispatch of thermal generating units under various power constraints in smart power grid," *Applied Soft Computing Journal*, vol. 58, pp. 401 – 426, 2017, cited by: 40. [Online]. Available: <https://www.scopus.com/pages/publications/85019598848?origin=resultslist>
- [15] T. Huang and A. Sanagavarapu Mohan, "A microparticle swarm optimizer for the reconstruction of microwave images," *IEEE Transactions on Antennas and Propagation*, vol. 55, no. 3 I, pp. 568 – 576, 2007, cited by: 51. [Online]. Available: <https://www.scopus.com/pages/publications/34047204586?origin=resultslist>
- [16] A. Biswas, S. Dasgupta, S. Das, and A. Abraham, "Synergy of pso and bacterial foraging optimization - a comparative study on numerical benchmarks," *Advances in Soft Computing*, vol. 44, pp. 255 – 263, 2007, cited by: 189. [Online]. Available: <https://www.scopus.com/pages/publications/61449110863?origin=resultslist>
- [17] M. Gonabadi, N. H. Hunt, and F. Fallahtafi, "EMG-based simulation for optimization of human-in-the-loop control in simple robotic walking assistance," *Journal of Sensor and Actuator Networks*, vol. 14, p. 113, 12 2025.
- [18] S. Duan, H. Luo, and H. Liu, "A multi-strategy seeker optimization algorithm for optimization constrained engineering problems," *IEEE Access*, vol. 10, pp. 7165–7195, 1 2022.
- [19] Z. Xia, A. Li, J. Li, H. Shi, M. Duan, and G. Zhou, "Model updating of an existing bridge with high-dimensional variables using modified particle swarm optimization and ambient excitation data," *Measurement: Journal of the International Measurement Confederation*, vol. 159, 2020, cited by: 49. [Online]. Available: <https://www.scopus.com/pages/publications/85082168705?origin=resultslist>
- [20] C. Turkeri, S. Ekinici, G. Yuksek, and D. Li, "A modified enzyme action optimizer-based FOPID controller for temperature regulation of a non-linear continuous stirred tank reactor," *Fractal and Fractional*, vol. 9, p. 811, 12 2025. [Online]. Available: <https://www.proquest.com/scholarly-journals/modified-enzyme-action-optimizer-based-fopid/docview/3286294597/se-2>
- [21] T. M. Shami, A. A. El-Saleh, M. Alswaitti, Q. Al-Tashi, M. A. Summakieh, and S. Mirjalili, "Particle swarm optimization: A comprehensive survey," *IEEE Access*, vol. 10, pp. 10 031–10 061, 1 2022.
- [22] N. F. Alkayem, M. Cao, L. Shen, R. Fu, and D. Šumarac, "The combined social engineering particle swarm optimization for real-world engineering problems: A case study

- of model-based structural health monitoring,” *Applied Soft Computing*, vol. 123, 2022, cited by: 40. [Online]. Available: <https://www.scopus.com/pages/publications/85130378745?origin=resultslist>
- [23] Z. Wang, Y. Li, L. Wu, and Q. Guo, “A nonlinear adaptive weight-based mutated whale optimization algorithm and its application for solving engineering problems,” *IEEE Access*, vol. 12, pp. 40 225–40 254, 1 2024.
- [24] X. Zhao, Q. Ma, J. Li, Z. Wu, H. Lu, and Y. Xiong, “Optimization design of lazy-wave dynamic cable configuration based on machine learning,” *Journal of Marine Science and Engineering*, vol. 13, p. 873, 4 2025. [Online]. Available: <https://www.mdpi.com/2077-1312/13/5/873>
- [25] A. Gálvez and A. Iglesias, “Particle swarm optimization for non-uniform rational b-spline surface reconstruction from clouds of 3d data points,” *Information Sciences*, vol. 192, pp. 174 – 192, 2012, cited by: 159. [Online]. Available: <https://www.scopus.com/pages/publications/84857864608?origin=resultslist>
- [26] K. Gopalakrishnan, “Neural network-swarm intelligence hybrid nonlinear optimization algorithm for pavement moduli back-calculation,” *Journal of Transportation Engineering*, vol. 136, no. 6, pp. 528 – 536, 2010, cited by: 42. [Online]. Available: <https://www.scopus.com/pages/publications/77955602655?origin=resultslist>
- [27] M. Z. b. Mohd Zain, J. Kanesan, J. H. Chuah, S. Dhanapal, and G. Kendall, “A multi-objective particle swarm optimization algorithm based on dynamic boundary search for constrained optimization,” *Applied Soft Computing Journal*, vol. 70, pp. 680 – 700, 2018, cited by: 109; All Open Access; Green Open Access. [Online]. Available: <https://www.scopus.com/pages/publications/85048989639?origin=resultslist>
- [28] S. Yao, D. Guo, Z. Sun, and G. Yang, “A modified multi-objective sorting particle swarm optimization and its application to the design of the nose shape of a high-speed train,” *Engineering Applications of Computational Fluid Mechanics*, vol. 9, no. 1, pp. 513 – 527, 2015, cited by: 43; All Open Access; Gold Open Access; Green Open Access. [Online]. Available: <https://www.scopus.com/pages/publications/84961328852?origin=resultslist>
- [29] Z. Cheng, C. Zhuo, and Q. Zhu, “Attitude control for hovering rocket decoy based on PSO-optimized sliding mode controller,” *Journal of Physics: Conference Series*, vol. 3126, p. 012064, 10 2025.
- [30] M. A. Sadi, A. Jamali, M. A. Zulkifli, M. N. Leman, S. Mohamaddan, and H. Hazmi, “Enhanced control strategies for lower limb rehabilitation robots: A comparative study of MRAC, PID-ZN, and MRAC-PSO controllers,” *Applied Bionics and Biomechanics*, vol. 2026, p. 8945626, 1 2026. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1155/abb/8945626>
- [31] F. Sancho Caparrini, “Pso: Optimización por enjambres de partículas,” <https://www.cs.us.es/~fsancho/Blog/posts/PSO.md.html>, 11 2019, consultado el 31 de mayo de 2026.
- [32] H. Dallaki, A. Sami, A. Hamzeh, and S. Hashemi, “Using feature selection for speed up hybrid pso/aco,” *JOURNAL OF COMPUTING*, vol. 3, pp. 2151–9617, 1 2011.
- [33] M. Sánchez, M. Duarte, L. Bárzaga, M. Orchard, and G. Ceballos, “Fractional gradient-based model reference adaptive control applied on an inverted pendulum-cart system,” *Fractal and Fractional*, vol. 9, p. 485, 7 2025.

- [34] N. Yasir, F. Mahmood, M. Siddeeq, and M. Jaffer, “Enhanced CNN hyperparameter tuning via swarm intelligence for soil type classification,” 8 2025.
- [35] S. M. Ebadi, E. Khamsehchi, and J. M. Kalatehno, “A comprehensive comparative analysis of particle swarm optimization and genetic algorithms in well acidizing optimization,” *Journal of Petroleum Exploration and Production Technology*, vol. 15, pp. 1–19, 3 2025.
- [36] I. N. Kassabalidis, M. A. El-Sharkawi, R. J. Marks II, L. S. Moulin, and A. P. A. Da Silva, “Dynamic security border identification using enhanced particle swarm optimization,” *IEEE Transactions on Power Systems*, vol. 17, no. 3, pp. 723 – 729, 2002, cited by: 101. [Online]. Available: <https://www.scopus.com/pages/publications/0036699876?origin=resultslist>
- [37] C. Castañeda, “AI-based optimization of a neural discrete-time sliding mode controller via bayesian, particle swarm, and genetic algorithms,” *Robotics*, vol. 14, p. 128, 9 2025.
- [38] A. Karamolachab, S. A. Mehdizadeh, and Y. Ampatzidis, “Design, optimization, and field evaluation of an automatic steering system for agricultural tractors using metaheuristic PID tuning,” *Agriculture*, vol. 16, p. 1004, 5 2026. [Online]. Available: <https://www.mdpi.com/2077-0472/16/9/1004>
- [39] S. Samal, S. K. Sarangi, and A. Sarangi, “Analysis of adaptive mutation in crazy particle swarm optimization,” in *2020 International Conference on Computational Intelligence for Smart Power System and Sustainable Energy (CISPSSE)*. IEEE, 7 2020, pp. 1–5. [Online]. Available: <https://ieeexplore.ieee.org/document/9212193/>